KEMENTERIAN PENDIDIKAN, KEBUDAYAAN, RISET, DAN TEKNOLOGI
POLITEKNIK NEGERI MALANG
JURUSAN TEKNOLOGI INFORMASI
Jl. Soekarno Hatta No. 9, Jatimulyo, Lowokwaru, Malang 65141
Telp. (0341) 404424 – 404425, Fax (0341) 404420
https://jti.polinema.ac.id

Mata Kuliah	:	Pemrograman Web Lanjut (PWL)
Program Studi	:	D4 – Teknik Informatika
Semester	:	4

Kelas	:	TI 2A
NIM	:	244107020240
Nama	:	Mochammad Rijal Dzaki Rifki Afifudin
Jobsheet Ke-	:	3

Laporan Jobsheet

1. Praktikum ke-1

Langkah	Jawaban/Deskripsi
1	Membuat database:
2	Setting .env

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:LF1yDctJKWUepI7gAtQiWzN9rZAPJ88XbzAlAhOKYaY=
APP_DEBUG=true
APP_URL=http://localhost

LOG_CHANNEL=stack
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug

DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=PWL_POS
DB_USERNAME=root
DB_PASSWORD=
```

2. Praktikum ke-2

Langkah	Jawaban/Deskripsi
1	Membuat migration:

```
INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_015258_create_m_level_table.php] created successfully.
```

Modifikasi kode:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_level', function(Blueprint $table) {
            $table->id('level_id');
            $table->string('level_kode', 10)->unique();
            $table->string('level_nama', 100);
```

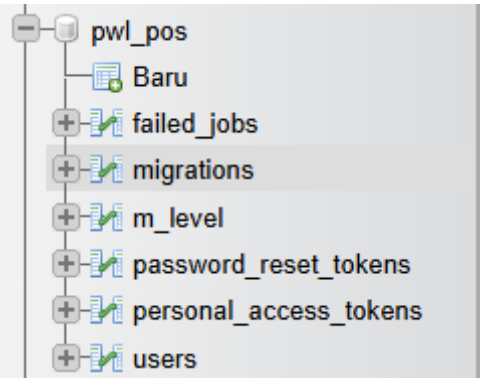
```
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_level');
    }
```

Simpan dengan cara:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan migrate

  INFO  Preparing database.


  INFO  Running migrations.

  2014_10_12_000000_create_users_table ........ 132ms DONE
  2014_10_12_100000_create_password_reset_tokens_table  53ms DONE
  2019_08_19_000000_create_failed_jobs_table ... 89ms DONE
  2019_12_14_000001_create_personal_access_tokens_table  97ms DONE
  2026_03_02_015258_create_m_level_table ....... 68ms DONE
```

2

Pengecekan pada phpMyAdmin:



3

Membuat migration

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan make:migration create_m_user_table

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_020137_create_m_user_table.php] created successfully.
```

Modifikasi kode:

```
<?php
```

```php
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_user', function (Blueprint $table) {
            $table->id('user_id');
            $table->unsignedBigInteger('level_id')->index();
            $table->string('username',20);
            $table->string('nama',100);
            $table->string('password',20);
            $table->timestamps();

            $table->foreign('level_id')->references('level_id')-
>on('m_level');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_user');
    }
};
```

Simpan dengan cara:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan m
igrate

  INFO  Running migrations.

  2026_03_02_020137_create_m_user_table ....... 263ms DONE
```

4	Membuat migration:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App>
                                                    php artisan make:migration create_m_kategori_table
>>
>> php artisan make:migration create_m_barang_table
>>
>> php artisan make:migration create_t_stok_table
>>
>> php artisan make:migration create_t_penjualan_table
>>
>> php artisan make:migration create_t_penjualan_detail_table

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_025706_create_m_kategori_table.php] created successfully.

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_025707_create_m_barang_table.php] created successfully.

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_025709_create_t_stok_table.php] created successfully.

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_025710_create_t_penjualan_table.php] created successfully.

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_025711_create_t_penjualan_detail_table.php] created successfully.
```

Modifikasi kode:

Kategori:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_kategori', function (Blueprint $table) {
            $table->id('kategori_id');
            $table->string('kategori_kode',10)->unique();
            $table->string('kategori_nama',100);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_kategori');
```

```
    }
};
```

Barang:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_barang', function (Blueprint $table) {
            $table->id('barang_id');
            $table->unsignedBigInteger('kategori_id')->index();
            $table->string('barang_kode',10)->unique();
            $table->string('barang_nama',100);
            $table->integer('harga_beli');
            $table->integer('harga_jual');
            $table->timestamps();

            $table->foreign('kategori_id')->references('kategori_id')-
>on('m_kategori');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_barang');
    }
};
```

Stok:

```
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('t_stok', function (Blueprint $table) {
            $table->id('stok_id');
            $table->unsignedBigInteger('barang_id')->index();
            $table->unsignedBigInteger('user_id')->index();
            $table->dateTime('stok_tanggal');
            $table->integer('stok_jumlah');
            $table->timestamps();

            $table->foreign('barang_id')->references('barang_id')-
>on('m_barang');
            $table->foreign('user_id')->references('user_id')-
>on('m_user');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('t_stok');
    }
};
```

Penjualan:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
```

```
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('t_penjualan', function (Blueprint $table) {
            $table->id('penjualan_id');
            $table->unsignedBigInteger('user_id')->index();
            $table->String('pembeli', 50);
            $table->String('penjualan_kode', 20)->unique();
            $table->dateTime('penjualan_tanggal');
            $table->timestamps();

            $table->foreign('user_id')->references('user_id')-
>on('m_user');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('t_penjualan');
    }
};
```

Penjualan detail:

```
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('t_penjualan_detail', function (Blueprint
$table) {
            $table->id('detail_id');
```

```
            $table->unsignedInteger('penjualan_id')->index();
            $table->unsignedInteger('barang_id')->index();
            $table->integer('harga');
            $table->integer('jumlah');
            $table->timestamps();

            $table->foreign('penjualan_id')-
>references('penjualan_id')->on('t_penjualan');
            $table->foreign('barang_id')->references('barang_id')-
>on('m_barang');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('t_penjualan_detail');
    }
};
```

simpan pada database dengan cara:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan migrate

  INFO  Running migrations.

  2026_03_02_025706_create_m_kategori_table ..... 188ms DONE
  2026_03_02_025707_create_m_barang_table ..... 315ms DONE
  2026_03_02_025709_create_t_stok_table ..... 701ms DONE
  2026_03_02_025710_create_t_penjualan_table ..... 658ms DONE
  2026_03_02_025711_create_t_penjualan_detail_table ..... 248ms DONE
```

3. Praktikum ke-3

Langkah	Jawaban/Deskripsi

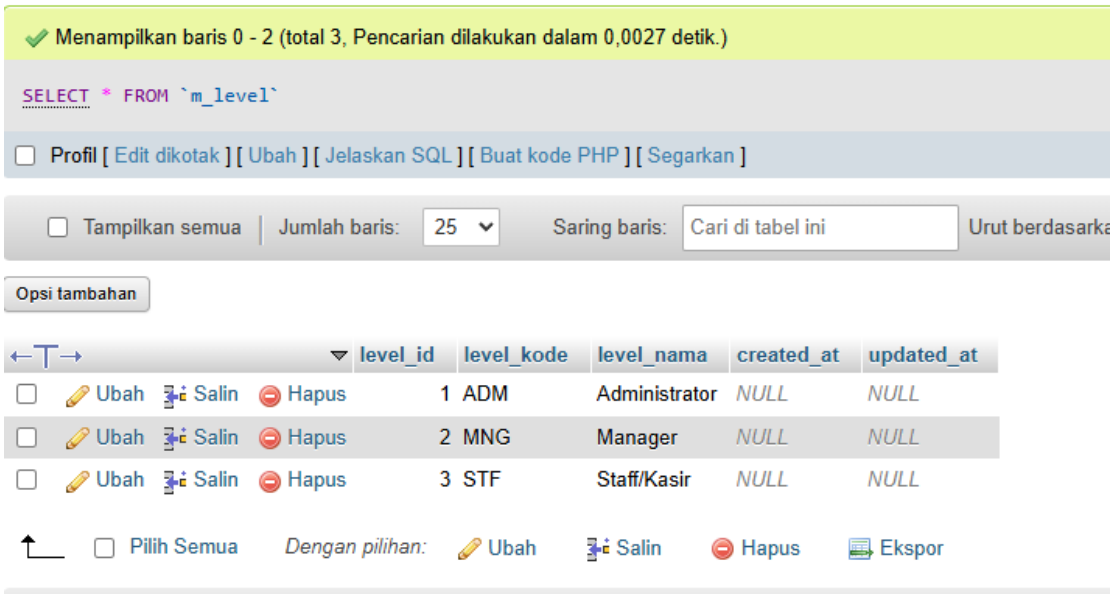
1	Membuat level seeder:
	`PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan make:seeder LevelSeeder`
	`INFO Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\LevelSeeder.php] created successfully.`
	Modifikasi kode:

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class LevelSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['level_id' => 1, 'level_kode' => 'ADM', 'level_nama' =>
'Administrator'],
            ['level_id' => 2, 'level_kode' => 'MNG', 'level_nama' =>
'Manager'],
            ['level_id' => 3, 'level_kode' => 'STF', 'level_nama' =>
'Staff/Kasir'],
        ];
        DB::table('m_level')->insert($data);
    }
}
```

Jalankan perintah berikut:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan d
b:seed --class=LevelSeeder

  INFO  Seeding database.
```

Pengecekan pada phpMyAdmin:



2

Membuat user seeder:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan m
ake:seeder UserSeeder

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\
database\seeders\UserSeeder.php] created successfully.
```

Modifikasi kode:

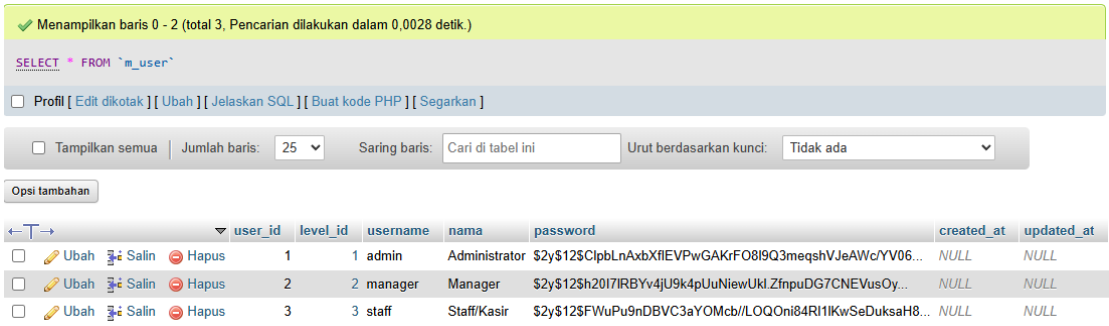
```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\Hash;
use Illuminate\Support\Facades\DB;

class UserSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            [
                'user_id' => 1,
                'level_id' => 1,
```

```
                'username' => 'admin',
                'nama' => 'Administrator',
                'password' => Hash::make('12345'),
            ],
            [
                'user_id' => 2,
                'level_id' => 2,
                'username' => 'manager',
                'nama' => 'Manager',
                'password' => Hash::make('12345'),
            ],
            [
                'user_id' => 3,
                'level_id' => 3,
                'username' => 'staff',
                'nama' => 'Staff/Kasir',
                'password' => Hash::make('12345'),
            ],
        ];
        DB::table('m_user')->insert($data);
    }
}
```

Jalankan perintah berikut:

```
>> php artisan db:seed --class=UserSeeder

  INFO  Seeding database.
```

Pengecekan pada phpMyAdmin:



3	Membuat table supplier terlebih dahulu:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan make:migration create_m_supplier_table

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_034102_create_m_supplier_table.php] created successf
ully.
```

Modifikasi kode:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('m_supplier', function (Blueprint $table) {
            $table->id('supplier_id');
            $table->string('supplier_kode', 10)->unique();
            $table->string('supplier_nama', 100);
            $table->string('supplier_alamat', 255);
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('m_supplier');
    }
};
```

Jalankan perintah berikut:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan make:migration add_supplier_id_to_m_barang_table --table=m_barang

  INFO  Migration [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\migrations/2026_03_02_034254_add_supplier_id_to_m_barang_table.php] create
d successfully.
```

Modikasi kode migration:

```php
<?php
```

```
use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::table('m_barang', function (Blueprint $table) {
            // Menambahkan kolom supplier_id setelah kategori_id
            $table->unsignedBigInteger('supplier_id')->index()-
>after('kategori_id');

            // Menambahkan Foreign Key
            $table->foreign('supplier_id')->references('supplier_id')-
>on('m_supplier');
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::table('m_barang', function (Blueprint $table) {
            $table->dropForeign(['supplier_id']);
            $table->dropColumn('supplier_id');
        });
    }
};
```

Membuat seeder:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan make:seeder KategoriSeeder
>> php artisan make:seeder SupplierSeeder
>> php artisan make:seeder BarangSeeder
>> php artisan make:seeder StokSeeder
>> php artisan make:seeder PenjualanSeeder
>> php artisan make:seeder PenjualanDetailSeeder

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\KategoriSeeder.php] created successfully.

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\SupplierSeeder.php] created successfully.

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\BarangSeeder.php] created successfully.

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\StokSeeder.php] created successfully.

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\PenjualanSeeder.php] created successfully.

  INFO  Seeder [C:\laragon\www\14_PWL\Pertemuan 03\Pos-App\database\seeders\PenjualanDetailSeeder.php] created successfully.
```

Modifikasi program:

Kategori:

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class KategoriSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            ['kategori_id' => 1, 'kategori_kode' => 'K01',
'kategori_nama' => 'Makanan'],
            ['kategori_id' => 2, 'kategori_kode' => 'K02',
'kategori_nama' => 'Minuman'],
            ['kategori_id' => 3, 'kategori_kode' => 'K03',
'kategori_nama' => 'Elektronik'],
            ['kategori_id' => 4, 'kategori_kode' => 'K04',
'kategori_nama' => 'Pakaian'],
            ['kategori_id' => 5, 'kategori_kode' => 'K05',
'kategori_nama' => 'Alat Tulis'],
        ];
        DB::table('m_kategori')->insert($data);
```

```
    }
}
```

Supplier:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class SupplierSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [
            [
                'supplier_id' => 1,
                'supplier_kode' => 'SUP01',
                'supplier_nama' => 'PT Makmur Jaya',
                'supplier_alamat' => 'Jl. Merdeka No. 1, Jakarta'
            ],
            [
                'supplier_id' => 2,
                'supplier_kode' => 'SUP02',
                'supplier_nama' => 'CV Sumber Rejeki',
                'supplier_alamat' => 'Jl. Sudirman No. 45, Bandung'
            ],
            [
                'supplier_id' => 3,
                'supplier_kode' => 'SUP03',
                'supplier_nama' => 'Toko Serba Ada',
                'supplier_alamat' => 'Jl. Pahlawan No. 10, Surabaya'
            ],
        ];
        DB::table('m_supplier')->insert($data);
    }
}
```

Barang:

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class BarangSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [];
        for ($i = 1; $i <= 15; $i++) {
            $data[] = [
                'barang_id' => $i,
                'kategori_id' => ($i % 5) + 1,
                // Baris di bawah ini sekarang diaktifkan (tanpa //)
                'supplier_id' => floor(($i - 1) / 5) + 1,
                'barang_kode' => 'BRG' . str_pad($i, 2, '0',
STR_PAD_LEFT),
                'barang_nama' => 'Barang ' . $i,
                'harga_beli' => 10000,
                'harga_jual' => 15000,
            ];
        }
        DB::table('m_barang')->insert($data);
    }
}
```

Stok:

```php
<?php

namespace Database\Seeders;
```

```php
use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Carbon\Carbon;

class StokSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [];
        for ($i = 1; $i <= 15; $i++) {
            $data[] = [
                'stok_id' => $i,
                'barang_id' => $i,
                'user_id' => 1, // Admin
                'stok_tanggal' => Carbon::now(),
                'stok_jumlah' => 100, // Tiap barang punya stok 100
            ];
        }
        DB::table('t_stok')->insert($data);
    }
}
```

Penjualan:

```php
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;
use Carbon\Carbon;

class PenjualanSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
```

```
        $data = [];
        for ($i = 1; $i <= 10; $i++) {
            $data[] = [
                'penjualan_id' => $i,
                'user_id' => 3, // Kasir
                'pembeli' => 'Pembeli ' . $i,
                'penjualan_kode' => 'PJN' . str_pad($i, 2, '0',
STR_PAD_LEFT),
                'penjualan_tanggal' => Carbon::now(),
            ];
        }
        DB::table('t_penjualan')->insert($data);
    }
}
```

Penjualan Detail:

```
<?php

namespace Database\Seeders;

use Illuminate\Database\Console\Seeds\WithoutModelEvents;
use Illuminate\Database\Seeder;
use Illuminate\Support\Facades\DB;

class PenjualanDetailSeeder extends Seeder
{
    /**
     * Run the database seeds.
     */
    public function run(): void
    {
        $data = [];
        $detail_id = 1;

        // 10 transaksi x 3 barang = 30 data
        for ($i = 1; $i <= 10; $i++) {
            for ($j = 1; $j <= 3; $j++) {
                $data[] = [
                    'detail_id' => $detail_id,
                    'penjualan_id' => $i,
                    'barang_id' => ($detail_id % 15) + 1, // ID barang
1-15 secara acak berurutan
                    'harga' => 15000, // Sama dengan harga jual
```

```
                    'jumlah' => 2, // Beli 2 qty per barang
                ];
                $detail_id++;
            }
        }
        DB::table('t_penjualan_detail')->insert($data);
    }
}
```

Jalankan perintah berikut untuk migration & seeder secara ulang

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan migrate:fresh --seed

  Dropping all tables ..... 816ms DONE

  INFO  Preparing database.

  Creating migration table ..... 66ms DONE

  INFO  Running migrations.

  2014_10_12_000000_create_users_table ..... 163ms DONE
  2014_10_12_100000_create_password_reset_tokens_table ..... 71ms DONE
  2019_08_19_000000_create_failed_jobs_table ..... 77ms DONE
  2019_12_14_000001_create_personal_access_tokens_table ..... 115ms DONE
  2026_03_02_015258_create_m_level_table ..... 93ms DONE
  2026_03_02_020137_create_m_user_table ..... 139ms DONE
  2026_03_02_025706_create_m_kategori_table ..... 68ms DONE
  2026_03_02_025707_create_m_barang_table ..... 141ms DONE
  2026_03_02_025709_create_t_stok_table ..... 227ms DONE
  2026_03_02_025710_create_t_penjualan_table ..... 138ms DONE
  2026_03_02_025711_create_t_penjualan_detail_table ..... 250ms DONE
  2026_03_02_034102_create_m_supplier_table ..... 127ms DONE
  2026_03_02_034254_add_supplier_id_to_m_barang_table ..... 161ms DONE
```

```
INFO  Seeding database.

Database\Seeders\LevelSeeder ..... RUNNING
Database\Seeders\LevelSeeder ..... 11 ms DONE

Database\Seeders\UserSeeder ..... RUNNING
Database\Seeders\UserSeeder ..... 866 ms DONE

Database\Seeders\KategoriSeeder ..... RUNNING
Database\Seeders\KategoriSeeder ..... 5 ms DONE

Database\Seeders\SupplierSeeder ..... RUNNING
Database\Seeders\SupplierSeeder ..... 6 ms DONE

Database\Seeders\BarangSeeder ..... RUNNING
Database\Seeders\BarangSeeder ..... 18 ms DONE

Database\Seeders\StokSeeder ..... RUNNING
Database\Seeders\StokSeeder ..... 18 ms DONE

Database\Seeders\PenjualanSeeder ..... RUNNING
Database\Seeders\PenjualanSeeder ..... 8 ms DONE

Database\Seeders\PenjualanDetailSeeder ..... RUNNING
Database\Seeders\PenjualanDetailSeeder ..... 8 ms DONE
```

4. Praktikum ke-4

Langkah	Jawaban/Deskripsi
1	Membuat level Controller:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-App> php artisan ma
ke:migration create_m_supplier_table ...
  INFO  Controller [C:\laragon\www\14_PWL\Pertemuan 03\Pos-A
pp\app\Http\Controllers\LevelController.php] created successf
ully.
```

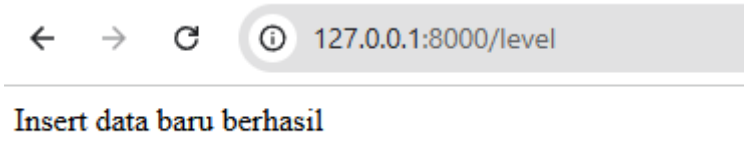
Modifikasi routes pada web.php:

```
Route::get('/level', [LevelController::class,'index']);
```

Modifikasi LevelController:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class LevelController extends Controller
{
    public function index()
    {
        DB::insert('insert into m_level(level_kode, level_nama,
created_at) values(?, ?, ?)', ['CUS', 'Pelanggan', now()]);

        return 'Insert data baru berhasil';
    }
}
```

Buka halaman /level



Pengecekan pada phpMyAdmin:

Menampilkan baris 0 - 3 (total 4, Pencarian dilakukan dalam 0,0057 detik.)

```
SELECT * FROM `m_level`
```

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua | Jumlah baris: 25 Saring baris: Cari di tabel ini Urut berdasarkan

Opsi tambahan

←T→	level_id	level_kode	level_nama	created_at	updated_at
Ubah Salin Hapus	1	ADM	Administrator	NULL	NULL
Ubah Salin Hapus	2	MNG	Manager	NULL	NULL
Ubah Salin Hapus	3	STF	Staff/Kasir	NULL	NULL
Ubah Salin Hapus	4	CUS	Pelanggan	2026-03-02 03:56:37	NULL

Modifikasi kode LevelController untuk mengupdate data:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode, level_nama,
created_at) values(?, ?, ?)', ['CUS', 'Pelanggan', now()]);
        // return 'Insert data baru berhasil';

        $row = DB::update('update m_level set level_nama = ? where
level_kode = ?', ['Customer', 'CUS']);
        return 'Update data berhasil, Jumlah data yang diupdate : ' .
$row. ' baris';
    }
}
```

Buka halaman /level:

127.0.0.1:8000/level

Update data berhasil, Jumlah data yang diupdate : 1 baris

Pengecekan pada phpMyAdmin:

Menampilkan baris 0 - 3 (total 4, Pencarian dilakukan dalam 0,0030 detik.)

```
SELECT * FROM `m_level`
```

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua | Jumlah baris: 25 Saring baris: Cari di tabel ini Urut berdas

Opsi tambahan

←T→	level_id	level_kode	level_nama	created_at	updated_at
Ubah Salin Hapus	1	ADM	Administrator	NULL	NULL
Ubah Salin Hapus	2	MNG	Manager	NULL	NULL
Ubah Salin Hapus	3	STF	Staff/Kasir	NULL	NULL
Ubah Salin Hapus	4	CUS	Customer	2026-03-02 03:56:37	NULL

Modifikasi kode, untuk menghapus data:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode, level_nama,
created_at) values(?, ?, ?)', ['CUS', 'Pelanggan', now()]);
        // return 'Insert data baru berhasil';

        // $row = DB::update('update m_level set level_nama = ? where
level_kode = ?', ['Customer', 'CUS']);
        // return 'Update data berhasil, Jumlah data yang diupdate :
' . $row. ' baris';

        $row = DB::delete('delete from m_level where level_kode = ?',
['CUS']);
```

```
        return 'Delete data berhasil. Jumlah data yang dihapus: ' .
$row. ' baris';
    }
}
```

Buka halaman /level:

Pengecekan pada phpMyAdmin:

Modifikasi kode, untuk melihat data:

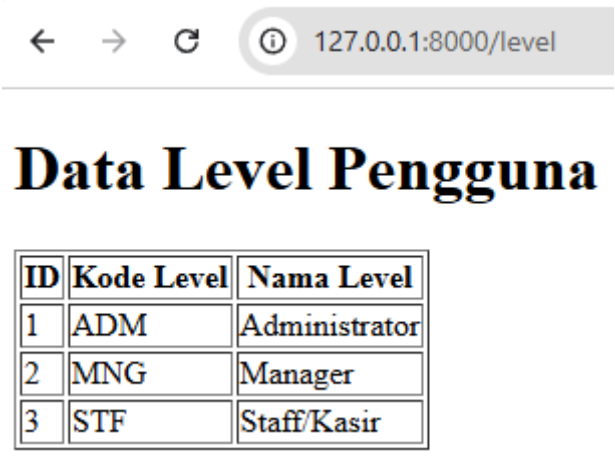
```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class LevelController extends Controller
{
    public function index()
    {
        // DB::insert('insert into m_level(level_kode, level_nama,
created_at) values(?, ?, ?)', ['CUS', 'Pelanggan', now()]);
        // return 'Insert data baru berhasil';
```

```
        // $row = DB::update('update m_level set level_nama = ? where
level_kode = ?', ['Customer', 'CUS']);
        // return 'Update data berhasil, Jumlah data yang diupdate :
' . $row. ' baris';

        // $row = DB::delete('delete from m_level where level_kode
= ?', ['CUS']);
        // return 'Delete data berhasil. Jumlah data yang dihapus: ' .
$row. ' baris';

        $data = DB::select('select * from m_level');
        return view('level', ['data' => $data]);
    }
}
```

Membuat file baru level.blade.php, dengan kode program:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Data Level Pengguna</title>
</head>
<body>
    <h1>Data Level Pengguna</h1>
    <table border="1" collpadding="2" collspacing="0">
        <tr>
            <th>ID</th>
            <th>Kode Level</th>
            <th>Nama Level</th>
        </tr>
        @foreach ( $data as $d )
            <tr>
                <td>{{ $d->level_id }}</td>
                <td>{{ $d->level_kode }}</td>
                <td>{{ $d->level_nama }}</td>
            </tr>
        @endforeach
    </table>
</body>
</html>
```

Buka halaman /level:



ID	Kode Level	Nama Level
1	ADM	Administrator
2	MNG	Manager
3	STF	Staff/Kasir

5. Praktikum ke-5

Langkah	Jawaban/Deskripsi
1	Membuat KategoriController:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos
-App> php artisan make:controller Kategor
iController
rs\KategoriController.php] created succes
sfully.
```

Setting route:

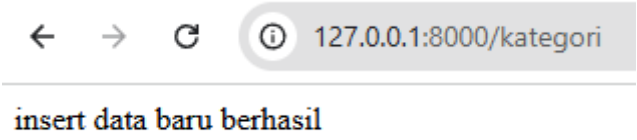
```
Route::get('/kategori', [KategoriController::class,'index']);
```

Modifikasi kode KategoriController:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class KategoriController extends Controller
{
    public function index()
        {
        $data = [
            'kategori_kode' => 'SMK',
```

```
            'kategori_nama' => 'Snack/Makanan Ringan',
            'created_at' => now()
        ];
        DB::table('m_kategori')->insert($data);
        return 'insert data baru berhasil';
        }
}
```

Buka halaman http://127.0.0.1:8000/kategori



insert data baru berhasil

Pengecekan database phpMyAdmin:

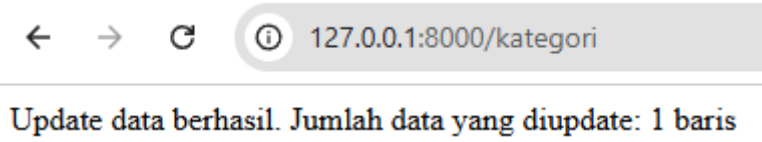
	kategori_id	kategori_kode	kategori_nama	created_at	updated_at
Ubah Salin Hapus	1	K01	Makanan	NULL	NULL
Ubah Salin Hapus	2	K02	Minuman	NULL	NULL
Ubah Salin Hapus	3	K03	Elektronik	NULL	NULL
Ubah Salin Hapus	4	K04	Pakaian	NULL	NULL
Ubah Salin Hapus	5	K05	Alat Tulis	NULL	NULL
Ubah Salin Hapus	6	SMK	Snack/Makanan Ringan	2026-03-02 04:30:16	NULL

Modifikasi kode, untuk mengupdate data:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class KategoriController extends Controller
{
```

```
    public function index()
    {
        // $data = [
        //     'kategori_kode' => 'SMK',
        //     'kategori_nama' => 'Snack/Makanan Ringan',
        //     'created_at' => now()
        // ];
        // DB::table('m_kategori')->insert($data);
        // return 'insert data baru berhasil';

        $row = DB::table('m_kategori')->where('kategori_kode', 'SMK')-
>update(['kategori_nama' => 'Camilan']);
        return 'Update data berhasil. Jumlah data yang diupdate: ' .
$row. ' baris';
    }
}
```

Bukan halaman http://127.0.0.1:8000/kategori



Pengecekan databas phpMyAdmin:

Modifikasi kode program, untuk menghapus data:

```
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class KategoriController extends Controller
{
    public function index()
    {
        // $data = [
        //     'kategori_kode' => 'SMK',
        //     'kategori_nama' => 'Snack/Makanan Ringan',
        //     'created_at' => now()
        // ];
        // DB::table('m_kategori')->insert($data);
        // return 'insert data baru berhasil';

        // $row = DB::table('m_kategori')->where('kategori_kode',
'SMK')->update(['kategori_nama' => 'Camilan']);
        // return 'Update data berhasil. Jumlah data yang diupdate: ' .
$row. ' baris';

        $row = DB::table('m_kategori')->where('kategori_kode', 'SMK')-
>delete();
        return 'Delete data berhasil. Jumlah data yang dihapus: ' .
$row. ' baris';
    }
}
```

Buka http://127.0.0.1:8000/kategori

Delete data berhasil. Jumlah data yang dihapus: 1 baris

Pengecekan database phpMyAdmin:

Menampilkan baris 0 - 4 (total 5, Pencarian dilakukan dalam 0,0133 detik.)

```
SELECT * FROM `m_kategori`
```

Profil [Edit dikotak] [Ubah] [Jelaskan SQL] [Buat kode PHP] [Segarkan]

Tampilkan semua | Jumlah baris: 25 Saring baris: Cari di tabel ini Urut berdasarkan

Opsi tambahan

←T→	kategori_id	kategori_kode	kategori_nama	created_at	updated_at
Ubah Salin Hapus	1	K01	Makanan	NULL	NULL
Ubah Salin Hapus	2	K02	Minuman	NULL	NULL
Ubah Salin Hapus	3	K03	Elektronik	NULL	NULL
Ubah Salin Hapus	4	K04	Pakaian	NULL	NULL
Ubah Salin Hapus	5	K05	Alat Tulis	NULL	NULL

Modifikasi kode, untuk melihat data:

```php
<?php

namespace App\Http\Controllers;

use Illuminate\Http\Request;
use Illuminate\Support\Facades\DB;

class KategoriController extends Controller
{
    public function index()
    {
        // $data = [
        //     'kategori_kode' => 'SMK',
        //     'kategori_nama' => 'Snack/Makanan Ringan',
        //     'created_at' => now()
        // ];
        // DB::table('m_kategori')->insert($data);
        // return 'insert data baru berhasil';

        // $row = DB::table('m_kategori')->where('kategori_kode',
'SMK')->update(['kategori_nama' => 'Camilan']);
        // return 'Update data berhasil. Jumlah data yang diupdate: ' .
$row. ' baris';

        // $row = DB::table('m_kategori')->where('kategori_kode',
'SMK')->delete();
```

```
        // return 'Delete data berhasil. Jumlah data yang dihapus: ' .
$row. ' baris';

        $data = DB::table('m_kategori')->get();
        return view('kategori',['data' => $data]);
    }
}
```

Membuat file baru yang bernama kategori.blade.php:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Data Kategori Pengguna</title>
</head>
<body>
    <h1>Data Kategori Pengguna</h1>
    <table border="1" collpadding="2" collspacing="0">
        <tr>
            <th>ID</th>
            <th>Kode Kategori</th>
            <th>Nama Kategori</th>
        </tr>
        @foreach ( $data as $d )
            <tr>
                <td>{{ $d->kategori_id }}</td>
                <td>{{ $d->kategori_kode }}</td>
                <td>{{ $d->kategori_nama }}</td>
            </tr>
        @endforeach
    </table>
</body>
</html>
```

Buka http://127.0.0.1:8000/kategori

ID	Kode Kategori	Nama Kategori
1	K01	Makanan
2	K02	Minuman
3	K03	Elektronik
4	K04	Pakaian
5	K05	Alat Tulis

6. Praktikum ke-6

Langkah	Jawaban/Deskripsi
1	Membuat UserModel:

```
p> php artisan make:model UserModel
uan 03\Pos-App\app\Models\UserModel.php] cre
ated successfully.
```

Modifikasi kode UserModel:

```php
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;

class UserModel extends Model
{
    use HasFactory;

    protected $table = 'm_user';
    protected $primaryKey = 'user_id';
}
```

Modifikasi routes:

```
Route::get('/user', [UserController::class,'index']);
```

Membuat UserController:

```
PS C:\laragon\www\14_PWL\Pertemuan 03\Pos-Ap
p> php artisan make:controller UserControlle
r
rController.php] created successfully.
```

Modifikasi UserController:

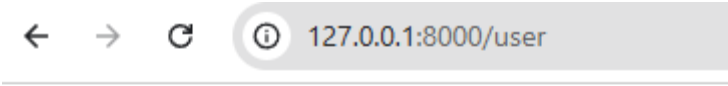
```
<?php

namespace App\Http\Controllers;

use App\Models\UserModel;
use Illuminate\Http\Request;

class UserController extends Controller
{
    public function index()
    {
        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

Membuat file baru yaitu user.blade.php

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-
scale=1.0">
    <title>Data User</title>
</head>
<body>
    <h1>Data User</h1>
    <table border="1" collpadding="2" collspacing="0">
        <tr>
            <th>ID</th>
            <th>Username</th>
```

```
            <th>Nama</th>
            <th>ID Level Pengguna</th>
        </tr>
        @foreach ( $data as $d )
            <tr>
                <td>{{ $d->user_id }}</td>
                <td>{{ $d->username }}</td>
                <td>{{ $d->nama }}</td>
                <td>{{ $d->level_id }}</td>
            </tr>
        @endforeach
    </table>
</body>
</html>
```

Buka http://127.0.0.1:8000/user



Data User

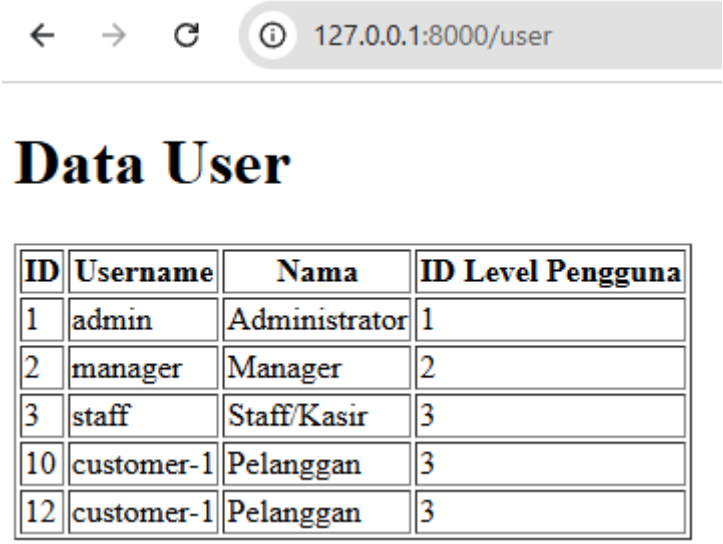
ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1
2	manager	Manager	2
3	staff	Staff/Kasir	3

Modifikasi kode UserController:

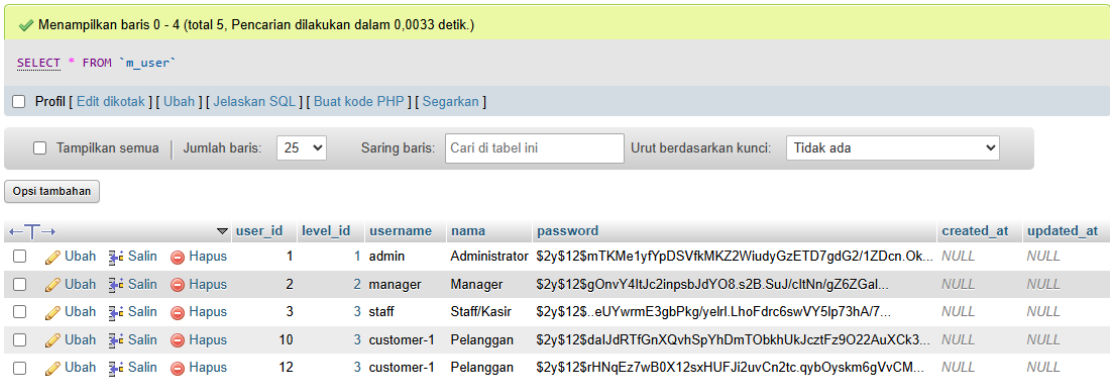
```
<?php

namespace App\Http\Controllers;

use App\Models\UserModel;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    public function index()
    {
```

```
        $data = [
            'username' => 'customer-1',
            'nama' => 'Pelanggan',
            'password' => Hash::make('12345'),
            'level_id' => 3
        ];
        UserModel::insert($data);

        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

Buka http://127.0.0.1:8000/user



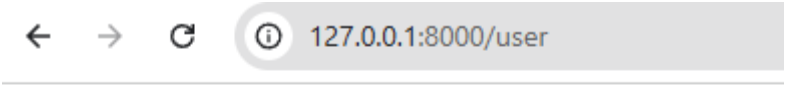
Pengecekan database phpMyAdmin:



Modifikasi UserController:

```php
<?php

namespace App\Http\Controllers;

use App\Models\UserModel;
use Illuminate\Http\Request;
use Illuminate\Support\Facades\Hash;

class UserController extends Controller
{
    public function index()
    {
        $data = [
            'nama' => 'Pelanggan Pertama',
        ];
        UserModel::where('username', 'customer-1')->update($data);

        $user = UserModel::all();
        return view('user', ['data' => $user]);
    }
}
```

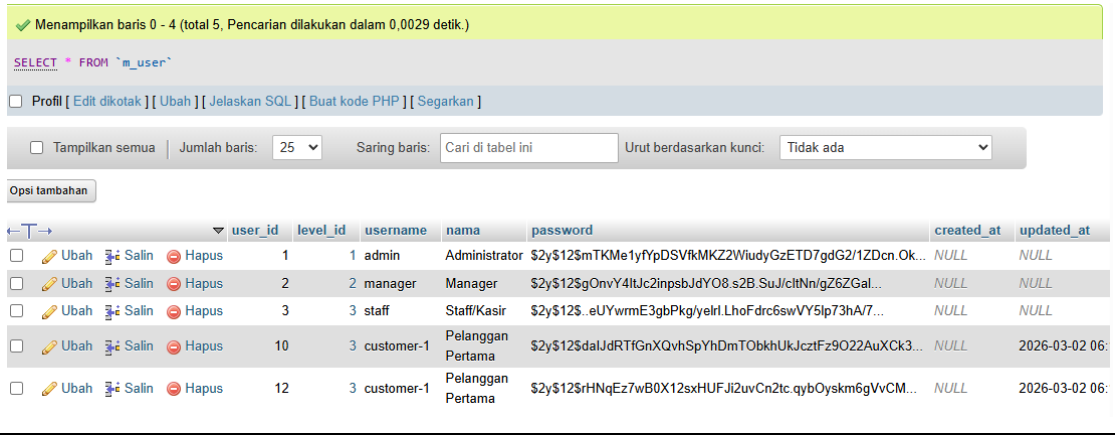
Buka http://127.0.0.1:8000/user



Data User

ID	Username	Nama	ID Level Pengguna
1	admin	Administrator	1
2	manager	Manager	2
3	staff	Staff/Kasir	3
10	customer-1	Pelanggan Pertama	3
12	customer-1	Pelanggan Pertama	3

Pengecekan database phpMyAdmin:



7. Jawaban Pertanyaan

Langkah	Jawaban/Deskripsi
1	`APP_KEY` adalah sebuah *string* acak yang digunakan oleh fitur enkripsi bawaan Laravel. Fungsinya untuk keamanan aplikasi, yaitu untuk mengenkripsi *session*, *cookie*, *password reset token*, dan data sensitif lainnya.
2	Dengan cara menjalankan perintah: php artisan key:generate
3	1. **`create_users_table`**: Membuat tabel `users` untuk menyimpan data akun pengguna aplikasi. 2. **`create_password_reset_tokens_table`**: Membuat tabel penampungan *token* sementara ketika ada *user* yang melakukan *request* "Lupa Password". 3. **`create_failed_jobs_table`**: Digunakan untuk menyimpan daftar proses/tugas *background* (*queue*) yang gagal dieksekusi oleh sistem. 4. **`create_personal_access_tokens_table`**: Tabel yang digunakan oleh fitur Laravel Sanctum untuk menyimpan *token* otentikasi jika kamu membuat API.
4	Tujuannya untuk *tracking* histori data; Laravel akan otomatis mengisi tanggal dan waktu saat ini pada kolom `created_at` ketika data baru ditambahkan, dan memperbarui nilai `updated_at` ketika data tersebut diedit.
5	Fungsi ini akan menghasilkan tipe data **`BIGINT UNSIGNED`** (dan secara otomatis memiliki atribut *Auto Increment* serta dijadikan sebagai *Primary Key*.
6	1. `$table->id();` akan membuat kolom *Primary Key* dengan nama default, yaitu **`id`**.

	2. `$table->id('level_id');` memodifikasi penamaan default tersebut, sehingga kolom *Primary Key* yang tercipta bernama **`level_id`**.
7	Tujuannya adalah untuk memastikan tidak boleh ada duplikasi data di dalam kolom tersebut. Jika ada data baru yang nilainya sama dengan data yang sudah ada, maka database akan menolaknya dan memunculkan *error*.
8	1. Di tabel `m_level`, `$table->id('level_id')` menciptakan *Primary Key* bertipe `BIGINT UNSIGNED`. 2. Di tabel `m_user`, `level_id` bertindak sebagai *Foreign Key*. Agar bisa direlasikan, kita tidak boleh memakai `$table->id()` lagi (karena tidak boleh ada dua *auto increment*), melainkan harus mendefinisikan tipe data yang cocok secara manual, yaitu menggunakan `$table->unsignedBigInteger('level_id')`.
9	mengubah teks *password* asli ("1234") menjadi string acak yang panjang dan tidak bisa dikembalikan ke bentuk aslinya. Tujuannya agar jika database diretas, *hacker* tetap tidak bisa membaca *password* asli pengguna.
10	Tanda tanya (`?`) berfungsi sebagai Parameter Binding (Placeholder). Ini adalah mekanisme keamanan standar (PDO) untuk mencegah serangan *SQL Injection*.
11	1. `protected $table = 'm_user';` berfungsi untuk mendefinisikan nama tabel spesifik yang akan digunakan oleh model tersebut. 2. `protected $primaryKey = 'user_id';` berfungsi untuk mendefinisikan nama *primary key* dari tabel yang digunakan.
12	Secara umum di dunia kerja Laravel, **Eloquent ORM** adalah yang paling mudah dan paling direkomendasikan. • **Alasannya:** Eloquent memperlakukan data sebagai *Object* PHP. Kodenya jauh lebih pendek, bersih, dan mudah dibaca (contoh: `UserModel::all()` langsung mengambil semua data). Eloquent juga sangat sakti dalam menangani relasi antar tabel (cukup panggil `$user->level->level_nama`). • **Query Builder** berada di tengah-tengah; lebih aman dan rapi dari SQL mentah, tapi agak panjang saat ada *join* tabel. • **DB Facade (Raw Query)** adalah yang paling "kaku" karena kamu harus mengetik manual syntax SQL. Biasanya hanya digunakan untuk *query* pelaporan yang sangat rumit dan spesifik.